RPC in Go

- Go standard library supports RPC right out-of-the-box
  - Package `net/rpc` https://pkg.go.dev/net/rpc
  - Provides access to the exported methods of an object across a network
- TCP or HTTP as "transport" protocols
- Requirements for server RPC methods
  - Method type and method are exported (capital letter)
  - Only two arguments, both exported
  - Second argument is a pointer to a reply struct that stores the corresponding data
  - An error is always returned

```
func (t *T) MethodName(argType T1, replyType *T2) error
```


RPC in Go: server

- On server side
  - Create a TPC server (or an HTTP server) to receive data
  - Use **`Register`** (or **`RegisterName`**): register an object, making it visible as a service

```
func (server *Server) Register(rcvr any) error
func RegisterName(name string, rcvr any) error
```

    - One input parameter, which is the interface: `any` is an alias for `interface{}`
    - It publishes the methods that are part of the given interface on the RPC server and allows them to be called by clients connecting to the service
  - Use **`Listen`** to announce on the local network address

```
func Listen(network, address string) (Listener, error)
```

RPC in Go: server

- Use **`Accept`** to accept connections on the listener and serve requests for each incoming connection

```
func (server *Server) Accept(lis net.Listener)
```

  - `Accept` blocks; if the server wishes to do other work as well, it should call this in a goroutine (go statement)

- Can also use HTTP handler for RPC messages
  - See example on course site

RPC in Go: client

- On client side
  - Use **`Dial`** to connect to RPC server at the specified network address (and port)

```
func Dial(network, address string) (*Client, error)
```

    - Use `DialHTTP` for HTTP connection
  - Use **`Call`** to call synchronous RPC: `Call` waits for the remote call to complete

```
func (client *Client) Call(serviceMethod string, args
any, reply any) error
```

  - Use **`Go`** to call asynchronous RPC: `Go` invokes the call asynchronously and signals completion using the `Call` structure's `Done` channel

```
func (client *Client) Go(serviceMethod string, args
any, reply any, done chan *Call) *Call
```

RPC in Go: client

- On client side
  - Struct **`Call`** represents an active RPC

```
type Call struct {
    ServiceMethod string  // The name of the service and method
                          // to call.
    Args any // The argument to the function (*struct)
    Reply any // The reply from the function (*struct)
    Error error // After completion, the error status.
    Done chan *Call // Receives *Call when Go is complete.
}
```


RPC in Go example: calculator

- Let's consider a simple RPC calculator with two functions: multiply and divide two integers
- Code available on course site

RPC in Go: synchronous call

- Need some setup in advance of this…
- `Call` makes blocking RPC call
- `Call` invokes the remote function, waits for it to complete, and returns its error status

```
// Synchronous call
args := &server.Args{7,8}
var reply int
err = client.Call("Arith.Multiply", args, &reply)
if err != nil {
   log.Fatal("arith error:", err)
}
fmt.Printf("Arith: %d*%d=%d", args.A, args.B, reply)
```

```
func (client *Client) Call(serviceMethod string,
args any, reply any) error
```


RPC in Go: asynchronous call

- How to make asynchronous RPC? Go uses a channel as parameter to retrieve RPC reply when the call is complete
- Done channel will signal when the call is complete by returning the same object of `Call`
  - If Done is nil, Go will allocate a new channel

```
// Asynchronous call
quotient := new(Quotient)
divCall := client.Go("Arith.Divide", args, quotient, nil)
divCall = <- divCall.Done
// check errors, print, etc.
```

```
func (client *Client) Go(serviceMethod string, args any,
reply any, done chan *Call) *Call
```

- For Go internal implementation, see https://go.dev/src/net/rpc/client.go?s=8029:8135 - L284

Go's RPC semantics

- Go's RPC is at-most-once
- Client opens TCP connection and writes request
  - TCP may retransmit but server's TCP receiver will filter out duplicates internally, with sequence numbers
  - No request retry in Go's RPC code (i.e. will not create a second TCP connection)
- However, client returns error if it does not get reply
  - Perhaps after TCP timeout
  - Perhaps server did not see request
  - Perhaps server processed request but server or network failed before reply came back

RPC in Go: marshaling and unmarshaling

- By default Go uses package `encoding/gob` for parameters marshaling (encode) and unmarshaling (decode) https://pkg.go.dev/encoding/gob
  - Package `gob` manages streams of gobs (Go binary values) exchanged between Encoder (transmitter) and Decoder (receiver)
  - A stream of gobs is *self-describing*: each data item in the stream is preceded by a specification of its type, expressed in terms of a small set of predefined types; pointers are not transmitted, but values they point to are transmitted
  - Basic usage: create encoder, transmit values, receive them with decoder
  - Requires that RPC client and server are both written in Go

RPC in Go: marshaling and unmarshaling

- Alternatives to `gob`

1. `net/rpc/jsonrpc` package in Go's standard library
   https://pkg.go.dev/net/rpc/jsonrpc
   – Implements a JSON-RPC 1.0 ClientCodec and ServerCodec for `rpc` package

2. gRPC

RPC in Go example: Word count

- Let's consider a basic word count: given a document, count the occurrences of each word in that document
  – Tokenize and put words in a hash map
- Code available on course site

```
"hello I am good hello bye bye Bye"

Result: map[am:1 bye:3 good:1 hello:2 i:1]
```

From word count to MapReduce

- Word count: Tokenize and put words in a hash map
- How do we parallelize this?

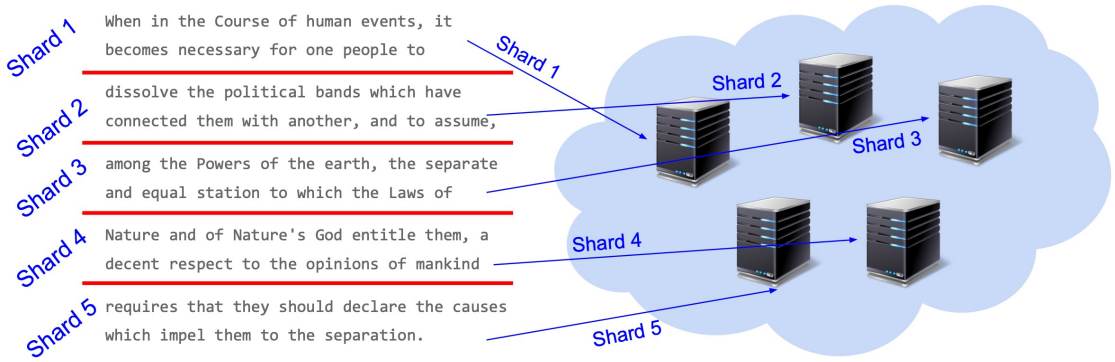
- Partition document into *n* partitions
- Build *n* hash maps, one for each partition
- Merge the *n* hash maps (by key)

- *How do you do this in a distributed environment?*



From word count to MapReduce

- Partition input document into shards and assign shards to nodes



From word count to MapReduce

- Count words locally on each node



From word count to MapReduce

- How to merge results computed locally?
- A first solution: send everything to one node

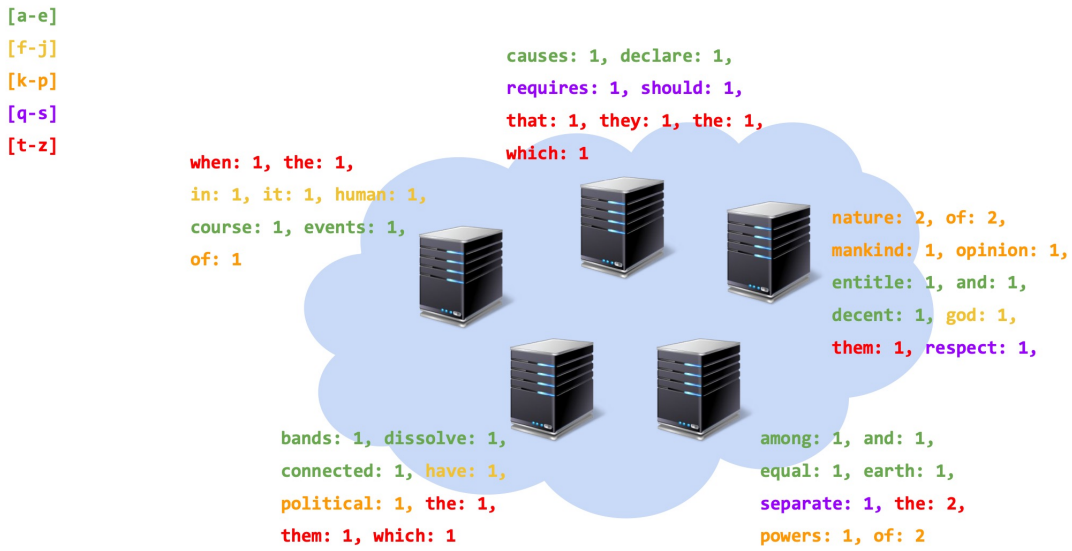
  What if data is too big? Too slow...

- The solution: partition key space among nodes (e.g, [a-e], [f-j], [k-p] ...)

1. Assign a key space to each node
2. Split local results by the key spaces
3. Fetch and merge results that correspond to the node's key space

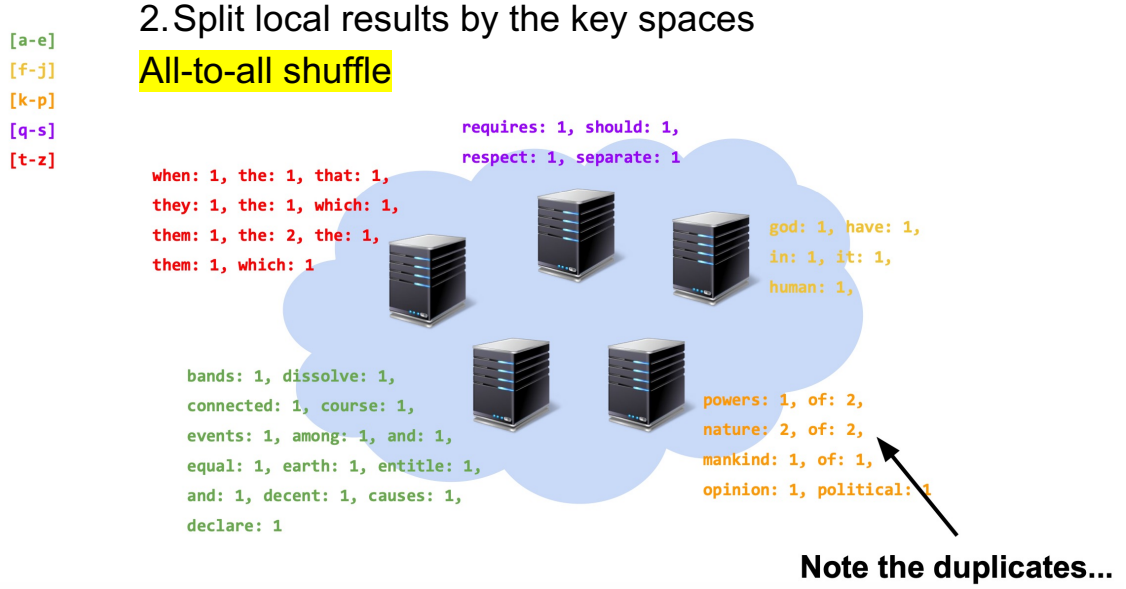
From word count to MapReduce

- The solution: partition key space among nodes
  1. Assign a key space to each node
  2. Split local results by the key spaces



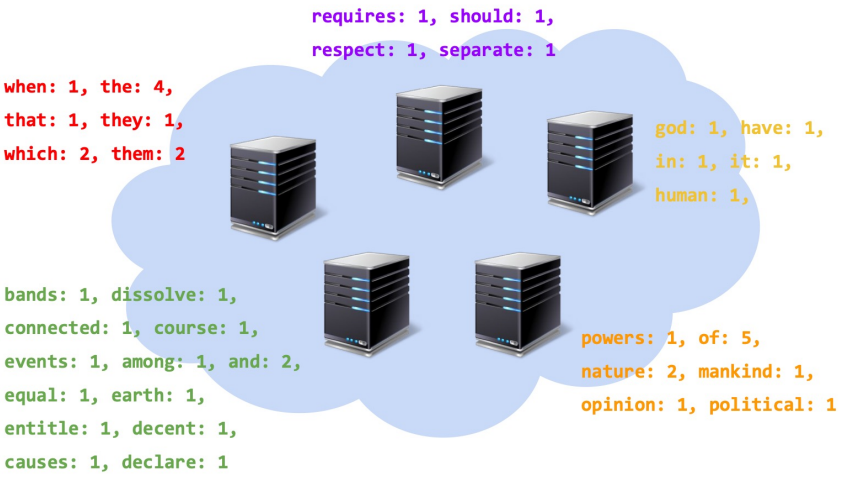
From word count to MapReduce

- The solution: partition key space among nodes
  1. Assign a key space to each node
  2. Split local results by the key spaces



From word count to MapReduce

- The solution: partition key space among nodes
  1. Assign a key space to each node
  2. Split local results by the key spaces
  3. Merge results received from other nodes



MapReduce in a nutshell

- Partition dataset into many shards
- **Map** stage: each node processes one or more shards locally
- **Reduce** stage: each node fetches and merges partial results from all other nodes

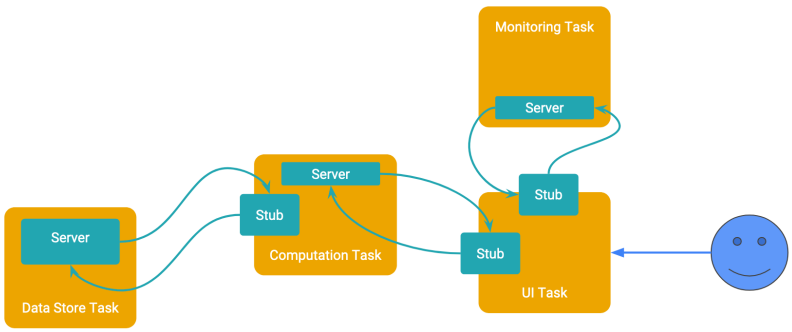
- But implementing MapReduce is not so easy…
  - Failures are common
  - Data skew causes unbalanced performance across cluster
  - System scale

Comparing RPC so far

- Let's compare RPC implementations
  - How do they differ in terms of distribution transparency?
  - Access transparency?
  - Location transparency?
  - Replication transparency?
    - Add server-side load balancer which acts as proxy between RPC clients and replicated RPC servers
  - Concurrency transparency?
  - Failure transparency?

Motivation for new RPC middleware

- Large-scale distributed applications composed of **microservices**
  - Microservices architecture: build sw application as a collection of *independent*, *autonomous* (developed, deployed, and scaled independently), *business capability–oriented*, and *loosely coupled* services
  - Multi-language (i.e., polyglot) development
  - Communication mainly structured as RPCs



gRPC



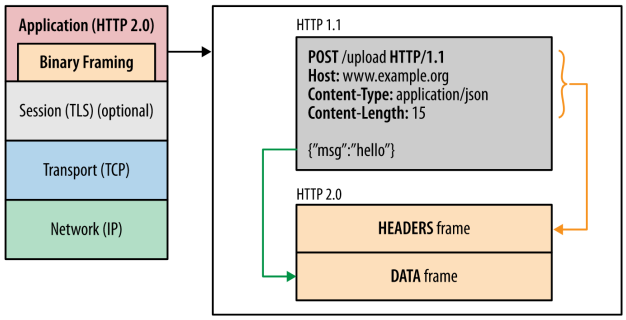
- High-performance, open source universal RPC framework https://grpc.io/
- Can run in any environment
  - Multi-language, multi-platform framework
- Main usage scenarios
  - Connect polyglot microservices that use request-reply communication style in and across data centers with pluggable support for load balancing, tracing, health checking and authentication
  - Connect devices, mobile apps and browsers to backend services
  - Generate efficient client libraries
- Developed by Google, now CNCF project
- Used by many companies and in many distributed systems
  - E.g., Cisco, Cockroach Labs, Netflix, Square, etcd https://etcd.io/

gRPC: main features

- **HTTP/2** for transport
  - Bidirectional streaming, multiplexing, header compression
- **Protocol buffers** as IDL
  - Simple service definition
  - Automatic code generation
  - Strict specification: prevents errors
- Plus authentication, flow control, blocking or non-blocking bindings, deadline/timeouts and cancellation
  - Deadline and timeouts allow clients to specify how long they are willing to wait for response

gRPC: HTTP/2

- Transport over HTTP/2
  - Basic idea of gRPC: treat RPCs as references to HTTP objects
- HTTP/2: major revision of HTTP that provides significant performance benefits over HTTP 1.x
- HTTP/2 in a nutshell
  - Binary framing layer: HTTP/2 request/response is divided into small messages and framed in binary format, making message transmission efficient



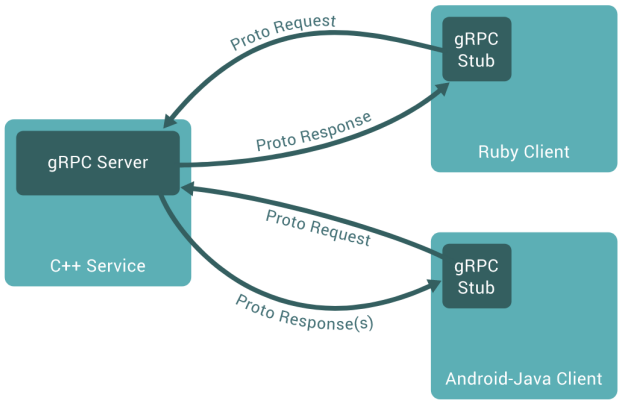
gRPC: HTTP/2

- HTTP/2 in a nutshell
  - From request/response messages to streams
    - Stream: bidirectional flow of bytes within an established connection, which may carry one or more messages
    - Message: complete sequence of frames that map to a logical request or response message
    - Frame: smallest unit of communication in HTTP/2, each containing a frame header, which at least identifies the stream to which the frame belongs
  - Request/response multiplexing (usage of a single connection per client): allows for efficient use of TCP connections and avoids head-of-line blocking at HTTP level
  - Native support for bidirectional streaming
  - HTTP header compression: to reduce protocol overhead

  See https://hpbn.co/http2/

gRPC: Protocol buffers

- gRPC uses protocol buffers (aka *protobufs*) as:
  - IDL to define service interface: automatic generation of client stubs and abstract server classes
  - Message interchange format: gRPC messages are serialized using protocol buffers, thus resulting in small message payloads
- Based on proxy pattern: *stub* and *server*



Protocol buffers

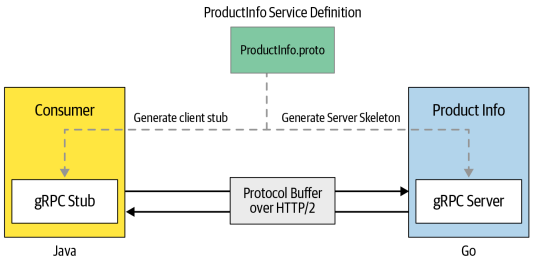
- Google's mature open-source mechanism to serialize structured data
- Binary data representation
- Strongly typed

```
message Person {
  string name = 1;
  int32 id = 2;
  bool has_ponycopter = 3;
}
```

- Data types are structured as messages
  - message: small logical record of information containing a series of name-value pairs called *fields*
  - Fields have unique field numbers (e.g., `string name = 1`), used to identify fields in message binary format

Protocol buffers: example

- ProductInfo service interface



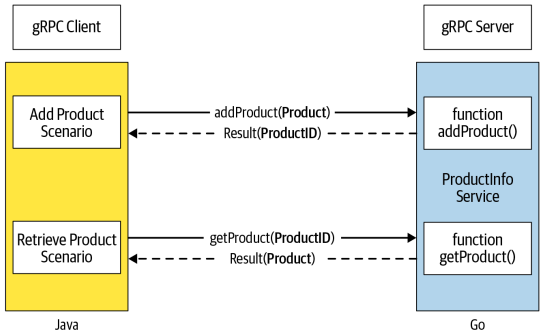

```
// ProductInfo.proto
syntax = "proto3";
package ecommerce;

service ProductInfo {
    rpc addProduct(Product) returns (ProductID);
    rpc getProduct(ProductID) returns (Product);
}

message Product {
    string id = 1;
    string name = 2;
    string description = 3;
}

message ProductID {
    string value = 1;
}
```

Client-server interaction



gRPC: basic steps

1. Define service (collection of remote methods) and message types that are exchanged between client and service in `.proto` file using protobufs as IDL
2. Generate server and client code using `protoc` (protocol buffer compiler) in your preferred language(s) from proto definition
   - Go: compile manually using `protoc` command
   - Java: use build automation tools (e.g., Maven, Gradle)
3. Use gRPC API in your preferred language (e.g., Go, Java, Python) to write service client and server
   - Let's consider Go: gRPC-Go https://grpc.io/docs/languages/go

gRPC: helloworld example in Go

- See https://grpc.io/docs/languages/go/quickstart/ and https://github.com/grpc/grpc-go/tree/master/examples/helloworld/

1. Define service (`helloworld.proto` file)

```
package helloworld;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```


gRPC: helloworld example in Go

2. Compile service definition:

```
$ protoc --go_out=. --go_opt=paths=source_relative \
    --go-grpc_out=. --go-grpc_opt=paths=source_relative \
    helloworld/helloworld.proto
```

- Automatically generated files:
  - `helloworld.pb.go`: contains protobuf code to populate, serialize, and retrieve request and response message types
  - `helloworld_grpc.pb.go`: contains
    - interface type (or *stub*) for clients to call with methods defined in Helloworld service
    - interface type for servers to implement, also with methods defined in Helloworld service
    - uses message definitions given in `helloworld.pb.go`
  - Take a look at those files and see that
    - message types have become Go structs
    - RPC definitions have become Go interfaces

gRPC: helloworld example in Go

3. Create server: composed of two parts
   a. Implement service interface generated from service definition: the actual “work”

```
func (s *server) SayHello(ctx context.Context,
   in *pb.HelloRequest) (*pb.HelloReply, error) {
    …
}
```

   b. Create and run gRPC server to listen for requests from clients and dispatch them to service implementation

```
lis, err := net.Listen("tcp", port)
if err != nil {
    log.Fatalf("failed to listen: %v", err)
}
s := grpc.NewServer()
pb.RegisterGreeterServer(s, &server{})
s.Serve(lis)
```


gRPC: helloworld example in Go

4. Create client
   – To call service methods, first create a *gRPC channel* to communicate with server using `Dial`

```
conn, err := grpc.Dial(address, opts…)
```

   – We need a client stub to perform RPCs: get it using `pb.NewGreeterClient` provided by `pb` package (generated from `.proto` file)

```
c := pb.NewGreeterClient(conn)
```

   – Call service methods on client stub: create and populate a request protobuf object (`HelloRequest`) and pass a `context` object which lets us change RPC’s behavior if necessary (e.g., time-out/cancel RPC in flight)

```
r, err := c.SayHello(ctx, &pb.HelloRequest{Name: name})
```

gRPC: update application components

- gRPC simplifies application updates
- If you want make changes to server API (e.g., adding new method)
  1. Update service definition (`.proto` file)
  2. Regenerate stubs so that client and server implementations can reflect changes
  3. Update server code to implement new method
  4. Update client code to call new method

gRPC: update helloworld example in Go

- Let's update gRPC service adding new method `SayHelloAgain()`
  1. Update `.proto` file
  2. Regenerate gRPC code using `protoc`
  3. Update server code to implement new method

```
func (s *server) SayHelloAgain(ctx context.Context, in
   *pb.HelloRequest) (*pb.HelloReply, error) {
    log.Printf("Received: %v", in.GetName())
    return &pb.HelloReply{Message: "Hello again " +
   in.GetName()}, nil
}
```

  4. Update client code to call new method

```
r, err = c.SayHelloAgain(ctx, &pb.HelloRequest{Name: name})
 if err != nil {
    log.Fatalf("could not greet: %v", err)
 }
 log.Printf("Greeting: %s", r.GetMessage())
```

gRPC: ProductInfo example

- Online retail having ProductInfo microservice which manages products and their information
- Clients can add and retrieve products

1. Define service in .proto file (see slide 128)
2. Implement server and client in Go
3. Implement server and client in Java

Go code: Teams/course website

Java code: https://github.com/grpc-up-and-running/samples/tree/master/ch02

See chapter 2 of gRPC: Up and Running
https://www.oreilly.com/library/view/grpc-up-and/9781492058328/

gRPC: types of RPC methods

- gRPC supports 4 kinds of service methods
  - Defined in `.proto` file
  - See routeguide example https://github.com/grpc/grpc-go/tree/master/examples/route_guide

1. Simple RPC: client sends request to server and waits for single response to come back (i.e., unary)
   - A normal function call

```
rpc SayHello (HelloRequest) returns (HelloReply) {}
```

2. Server-streaming RPC: client sends request to server and gets stream to read a sequence of messages back
   - Client reads from stream until there are no more messages
   - gRPC guarantees message ordering within individual RPC call

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```

gRPC: types of RPC methods

3. Client-streaming RPC: client writes sequence of messages and sends them to server, waits for server to read them and return its response
   - gRPC guarantees message ordering within individual RPC call

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

4. Bidirectional streaming RPC: both sides send sequence of messages using read-write stream (i.e., full duplex)
   - The two streams operate independently, so client and server can read and write messages as preferred (server can wait until it has received all client's messages before writing its messages, or server and client can play "ping-pong")
   - gRPC preserves message ordering in each stream

```
rpc RouteChat(stream RouteNote) returns (stream RouteNote) {}
```

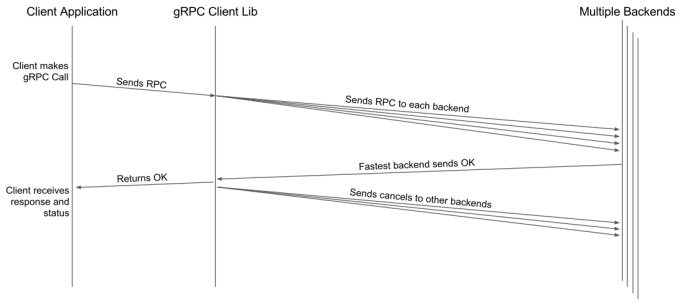

gRPC: distribution transparency

- What about distribution transparency?
  - Access transparency?
    - The usual proxy pattern
  - Location transparency?
    - gRPC supports service discovery mechanisms to locate services dynamically
    - By default, DNS as name resolver https://grpc.io/docs/guides/custom-name-resolution/
  - Concurrency transparency?
    - Language dependent, for Go see https://github.com/grpc/grpc-go/blob/master/Documentation/concurrency.md
  - Replication transparency?
    - gRPC can integrate with load balancers to distribute client requests evenly across multiple server instances https://grpc.io/docs/guides/custom-load-balancing

gRPC: distribution transparency

- What about distribution transparency?
  - Failure transparency?
    - At-most-once by default
    - gRPC can support implementing retry strategies for transient failures, including *exponential backoff* https://grpc.io/docs/guides/retry/
    - gRPC supports request hedging: sending multiple copies of same request without waiting for response https://grpc.io/docs/guides/request-hedging/

      See: Dean and Barroso, The Tail at Scale, 2013 https://research.google/pubs/the-tail-at-scale/



gRPC: other features

- Security
  - gRPC supports SSL/TLS
  - How to use
    - Need to generate SSL/TLS certificates (for development only: self-signed certificates using OpenSSL)
    - On server side: configure gRPC server to use SSL/TLS certificates
    - On client side. configure gRPC client to trust server's SSL/TLS certificates
    - Example: https://github.com/grpc/grpc-go/tree/master/examples/features/encryption

- Benchmarking and load testing
  - ghz tool https://ghz.sh

gRPC: weaknesses

- Limited support in browsers
  - Cannot directly call a gRPC service from a browser
  - gRPC-Web can provide gRPC support to browser but limited features (only simple RPC and limited server streaming) https://grpc.io/docs/platforms/web/basics/
- Non-human readable format
  - Protobuf is efficient to send and receive, but its binary format is not human readable
  - Developers need additional tools to analyze protobuf payloads on the wire, write manual requests, and perform debugging
    - E.g., gRPC command-line tool or Wireshark https://wiki.wireshark.org/gRPC

References

- Chapter 4 of van Steen & Tanenbaum book
- Remote Procedure Calls https://www.linuxjournal.com/article/2204
- Java Remote Method Invocation Specification https://docs.oracle.com/en/java/javase/23/docs/specs/rmi/
- Trail: RMI https://docs.oracle.com/javase/tutorial/rmi/
- RPyC tutorial https://rpyc.readthedocs.io/en/latest/tutorial.html
- RPyC documentation https://rpyc.readthedocs.io
- Go's package rpc https://pkg.go.dev/net/rpc
- RPC in the Go standard library, in Building Microservices with Go https://subscription.packtpub.com/book/web-development/9781786468666/1/ch01lvl1sec12/rpc-in-the-go-standard-library
- gRPC - Up and Running https://www.oreilly.com/library/view/grpc-up-and/9781492058328/